AIAC ASSIGNMENT-13.5

Name:K.Lokesh **2303A51201, B21**

Task Description #1 (Refactoring – Removing Global Variables)

• Task: Use AI to eliminate unnecessary global variables from the code.

• Instructions:

o Identify global variables used across functions.

o Refactor the code to pass values using function parameters.

o Improve modularity and testability.

• Sample Legacy Code:

```
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
```

• Expected Output:

o Refactored version passing rate as a parameter or using a configuration structure.

```python
def calculate_interest(amount, rate):
    return amount * rate
# Example usage
rate = 0.1
print(calculate_interest(1000, rate))
# Alternatively, using a configuration structure
class Config:
    def __init__(self, rate):
        self.rate = rate
def calculate_interest_with_config(amount, config):
    return amount * config.rate
# Example usage with configuration structure
config = Config(rate=0.1)
print(calculate_interest_with_config(1000, config))
```

Output:

```
100.0
100.0
```

Task Description #2 : (Refactoring Deeply Nested Conditionals)

• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

• Focus Areas:

o Readability

o Logical simplification

o Maintainability

Legacy Code:

score = 78

if score >= 90:

```
print("Excellent")
else:
if score >= 75:
print("Very Good")
else:
if score >= 60:
print("Good")
else:
print("Needs Improvement")
```

Expected Outcome:

o Flattened logic using guard clauses or a mapping-based approach.

```python
# Test case 1: score = 95
score = 95
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:
    print("Needs Improvement")
# Test case 2: score = 80
score = 80
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:
    print("Needs Improvement")
```

```python
# Alternatively, using a mapping-based approach
score = 78
grade_mapping = {
    (90, float('inf')): "Excellent",
    (75, 90): "Very Good",
    (60, 75): "Good",
    (0, 60): "Needs Improvement"
}
for (lower, upper), grade in grade_mapping.items():
    if lower <= score < upper:
        print(grade)
        break
```

Output:

```
Very Good
Excellent
Very Good
```

Task3: Use AI to refactor repeated file open/read/close logic.

• Focus Areas:

o DRY principle

o Context managers

o Function reuse

Legacy Code:

f = open("data1.txt")

print(f.read()) f.close()

f = open("data2.txt")

print(f.read()) f.close()

Expected Outcome:

o Reusable function using with open() and parameters.

```python
def read_file(file_name):
    with open(file_name, 'r') as f:
        content = f.read()
    return content
print(read_file("data1.txt"))
print(read_file("data2.txt"))
```

Task 4 :(Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False for

u in users:

if u == name:

found = True

print("Access Granted" if found else "Access Denied") Expected Outcome:

o Use of sets or dictionaries with complexity justification.

```
# Refactored Code using a Set for O(1) average time complexity
users = {"admin", "guest", "editor", "viewer"}# Using a set for O(1) average time complexity
name = input("Enter username: ")
if name in users:  # Checking membership in a set is O(1) on average
    print("Access Granted")
else:
    print("Access Denied")
# Justification:
# Using a set allows for O(1) average time complexity for membership checks,
# which is more efficient than the O(n) time complexity of a linear search through a list.
```

```
Enter username: admin
Access Granted
```

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

• Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

```
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
```

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

```python
class EmployeeSalaryCalculator:
    def __init__(self, salary):
        self.salary = salary

    def calculate_tax(self):
        return self.salary * 0.2

    def calculate_net_salary(self):
        tax = self.calculate_tax()
        return self.salary - tax
# Example usage
employee = EmployeeSalaryCalculator(50000)
net_salary = employee.calculate_net_salary()
print(net_salary)
```

```
40000.0
```

Task 6 (Refactoring for Performance Optimization)

• Task: Use AI to refactor a performance-heavy loop handling large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

total = 0

for i in range(1, 1000000): if i % 2 == 0:

total += i

print(total)

Expected Outcome:

o Optimized logic using mathematical formulas or comprehensions, with time comparison.

```
# Refactored Code using a Mathematical Formula for the Sum of Even Numbers
n = 999998  # Largest even number less than 1,000,000
total = n * (n + 2) // 4  # Using the formula for the sum of the first k even numbers: k * (k
print(total)
```

Output:

```
249999500000
```

Task 7 (Removing Hidden Side Effects)

• Task: Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring

o Predictability

Legacy Code:

```
data = []
def add_item(x):
data.append(x)
add_item(10)
add_item(20)
print(data)
```

Expected Outcome:

o Refactored function returning values instead of mutating

globals.

```
def add_item(x, data):
    return data + [x]
data = []
data = add_item(10, data)
data = add_item(20, data)
print(data)
```

Task 8 (Refactoring Complex Input Validation Logic)

• Task: Use AI to simplify and modularize complex validation rules.

• Focus Areas:

o Readability

o Testability

Legacy Code:

password = input("Enter password: ") if

len(password) >= 8:

if any(c.isdigit() for c in password): if

any(c.isupper() for c in password):

print("Valid Password")

else:

print("Must contain uppercase") else:

print("Must contain digit") else:

print("Password too short")

Expected Outcome:

o Separate validation functions with clear responsibility

```
def is_valid_length(password):
    return len(password) >= 8

def contains_digit(password):
    return any(c.isdigit() for c in password)

def contains_uppercase(password):
    return any(c.isupper() for c in password)

def is_valid_password(password):
    return is_valid_length(password) and contains_digit(password) and contains_uppercase(passw

password = input("Enter password: ")
if is_valid_password(password):
    print("Valid Password")
else:
    if not is_valid_length(password):
        print("Password too short")
    elif not contains_digit(password):
        print("Must contain digit")
    elif not contains_uppercase(password):
        print("Must contain uppercase")
```

Output:

```
Enter password: jvhc
Password too short
```